

Motion Planning con Potenziale Artificiale

DI LUCA CALLEGARI

1 Introduzione all'idea

Il codice presentato simula il movimento di un robot mobile in un ambiente bidimensionale, nel quale deve raggiungere una destinazione (il **goal**) evitando collisioni con degli **ostacoli** statici. Per ottenere questo comportamento in modo naturale, senza specificare a mano un percorso ottimale, viene impiegato un metodo noto come **campo di potenziale artificiale**.

L'idea chiave è di trattare il robot come una particella che si muove in un campo di forze: viene **attratto dal goal** come se fosse una carica positiva verso una carica negativa, e **respinto dagli ostacoli** come se fossero cariche dello stesso segno. Sommando queste forze, si ottiene un campo di potenziale che guida dinamicamente il robot verso la meta, evitando allo stesso tempo le aree pericolose.

2 Come funziona il codice

La simulazione è costruita in modo modulare e segue una logica precisa, basata su fisica semplificata e calcolo numerico.

2.1 Modellazione del potenziale

Viene definito un **potenziale attrattivo** che cresce con la distanza dal goal. Questo potenziale è una semplice parabola centrata sul punto di arrivo, il che garantisce che il minimo globale del campo di potenziale si trovi esattamente dove si vuole che il robot arrivi.

(*Funzione potenziale attrattiva*)

```
PhiAttr[x_, y_] := (1/2)*alpha*Norm[{x, y} - goal]^2;
```

Osserviamo che questa funzione prende come argomento la posizione corrente del robot, espressa dalle coordinate (x, y) , e restituisce un valore numerico sempre positivo. Inoltre, si è scelto di utilizzare la definizione con l'operatore `:=` per fare in modo che, ogni volta che la funzione viene chiamata, il valore del potenziale attrattivo venga ricalcolato dinamicamente in base alla posizione attuale.

Accanto a questo, c'è un **potenziale repulsivo** centrato su ciascun ostacolo. Questo potenziale è nullo se il robot si trova al di fuori di una certa distanza `d0` dall'ostacolo, ma cresce rapidamente all'avvicinarsi di esso. In questo modo, il robot è spinto via prima di toccarlo.

(*Funzione potenziale repulsiva*)

```
PhiRep[x_, y_] := Total[Table[Module[{d = Norm[{x, y} - o]},  
  If[d < d0, (1/2)*beta*(1/d^2 - 1/d0^2)^2, 0]],  
  {o, obstacles}]];
```

Entrambi i potenziali sono scalati da coefficienti `alpha` e `beta` che ne regolano l'intensità.

Il **potenziale totale** è dato dalla somma di quello attrattivo e di tutti i contributi repulsivi. La traiettoria del robot viene calcolata seguendo il **gradiente negativo del potenziale**, ovvero la direzione in cui il potenziale decresce più rapidamente.

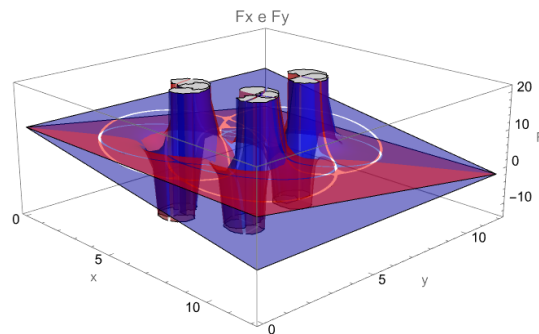
2.2 Calcolo del campo vettoriale

Il campo di forze che muove il robot viene derivato numericamente dal potenziale, usando una differenza finita per calcolare le derivate parziali.

```
(*Campo vettoriale:gradiente numerico del potenziale*)
vecField[{x_,y_}] := -{N[(Phi[x + 0.001, y] - Phi[x - 0.001, y])/0.002],
  N[(Phi[x, y + 0.001] - Phi[x, y - 0.001])/0.002]};
```

Questo campo vettoriale viene poi visualizzato tramite `VectorPlot`, con frecce colorate che ne indicano la direzione e l'intensità.

Utilizzando la funzione `Plot3D` su entrambe le componenti del campo vettoriale restituite da `vecField`, ho rappresentato separatamente le due funzioni in un unico grafico tridimensionale, combinandole tramite `Show`. In questo modo è possibile visualizzare l'andamento spaziale di ciascuna componente (in direzione x e y) del campo.



Come si può vedere entrambi i piani convergono al **goal** in posizione (10.0,10.0).

2.3 Sicurezza convergenza al goal

Ci si può legittimamente chiedere perché, come campo vettoriale, sia stato scelto proprio l'antigradiente del potenziale. La risposta risiede nelle proprietà della funzione potenziale totale, che è una funzione quadratica strettamente convessa. Questo tipo di funzione ammette un unico minimo globale, che nel nostro caso coincide con la posizione del goal.

Durante il corso di Ottimizzazione per Big Data ho studiato che, in presenza di una funzione strettamente convessa, seguire l'antigradiente garantisce la convergenza verso il minimo globale. Pertanto, scegliendo come direzione di movimento l'antigradiente, siamo matematicamente certi che l'algoritmo condurrà il robot verso il goal.

2.4 Simulazione della traiettoria

La posizione iniziale del robot è fissata, e ad ogni passo esso si muove seguendo la direzione del gradiente, normalizzata per mantenere i passi uniformi. È presente anche una piccola componente aleatoria, modellata con `RandomReal`, che simula un comportamento più realistico e permette di evitare blocchi in punti di minimo locale poco profondi. La simulazione si arresta quando il robot si avvicina abbastanza al goal o se il gradiente si annulla (cioè se resta bloccato).

Durante il movimento, ogni posizione viene salvata tramite `Sow` e raccolta con `Reap`, generando così l'intera traiettoria.

2.5 Animazione e visualizzazione

Per creare l'animazione, viene generato un frame per ciascuna posizione del robot lungo la traiettoria. Ogni frame è costruito con `Show`, unendo:

- il campo vettoriale aggiornato;
- la traiettoria tracciata fino a quel punto, disegnata in nero;

- il robot nella sua posizione attuale, indicato con un punto verde;
- gli ostacoli, rappresentati come cerchi neri;
- il goal (in rosso) e il punto di partenza con le relative etichette.

Tutti i frame sono infine raccolti in una lista e passati a `ListAnimate`, che consente di visualizzare in sequenza il movimento del robot in tempo reale.

3 Conclusione

Il codice implementa un'animazione dinamica e visivamente informativa della navigazione robotica usando il concetto di potenziali artificiali. Grazie all'uso combinato di modellazione fisica semplificata, strumenti di visualizzazione di Mathematica e simulazione numerica, si ottiene un comportamento emergente credibile e facilmente estendibile. Questo approccio è didatticamente molto utile per mostrare principi di robotica, ottimizzazione e dinamica dei sistemi in ambienti ostili o complessi.